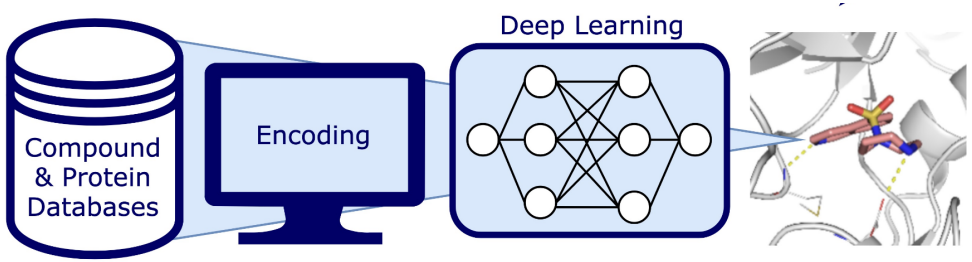
UNIVERSITÄT DES SAARLANDES

Data Driven Drug Design
Machine Learning with Python

Prof. Dr. Andrea Volkamer
Data Driven Drug Design Group



What we will do in detail …

Session 1

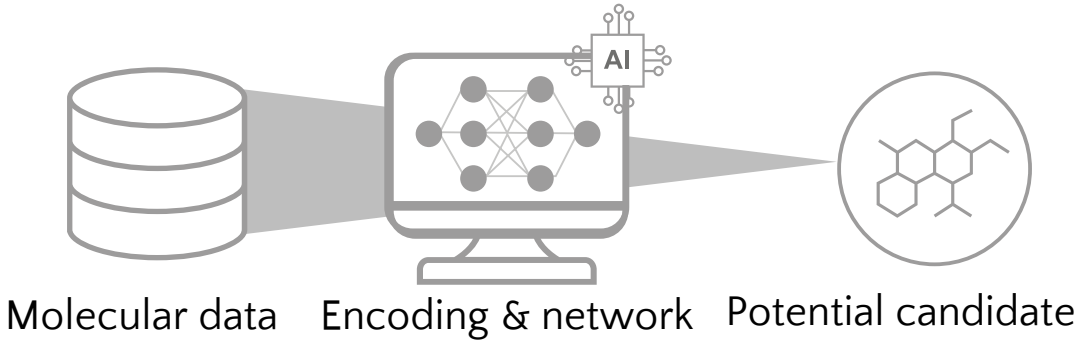
- CADD, Artificial Intelligence (AI) and Machine Learning (ML)
- The importance of data quality for AI/ML: Data preprocessing and analysis
- Molecular representations: Fingerprints and similarities

Session 2

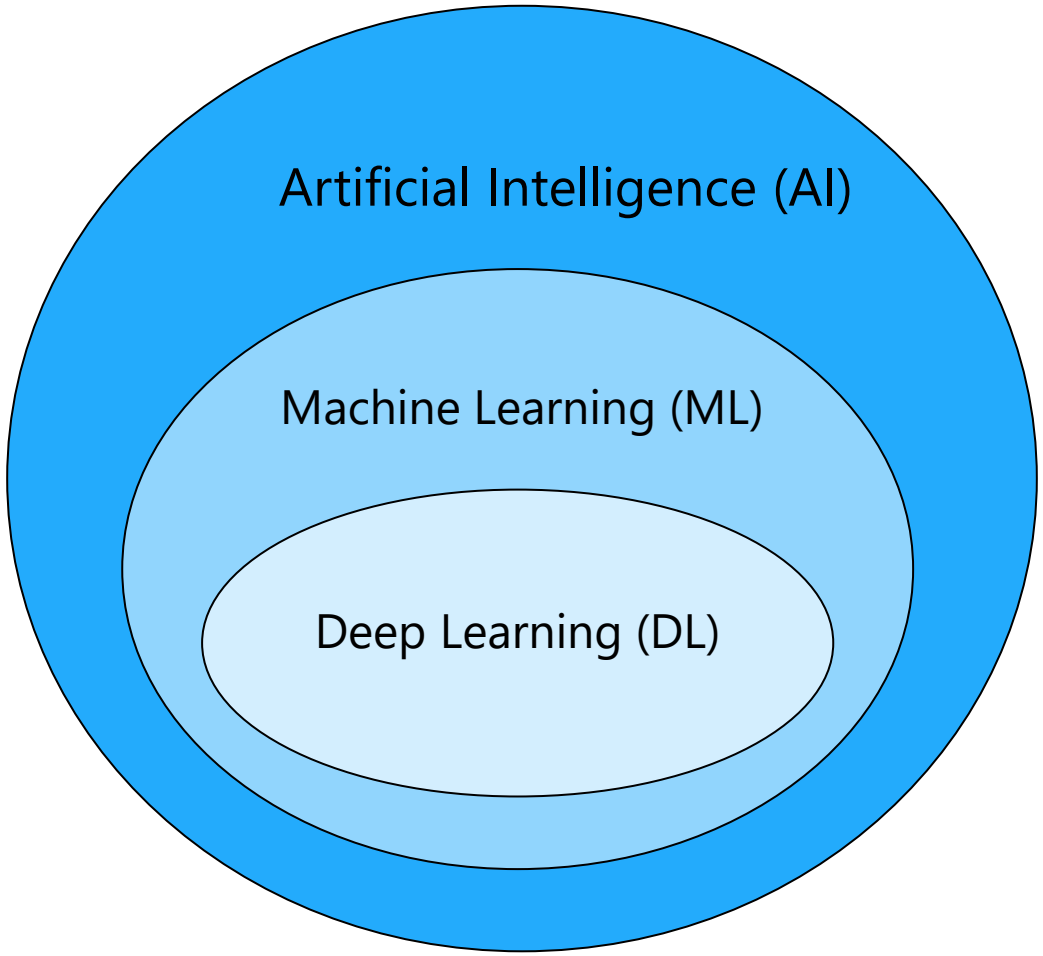
- Introduction machine learning (ML)
- (Un)supervised learning: RF, SVM, …
- Performance evaluation

Session 3

- **Deep learning**
- **Neuronal networks**
- Molecular encodings and architectures

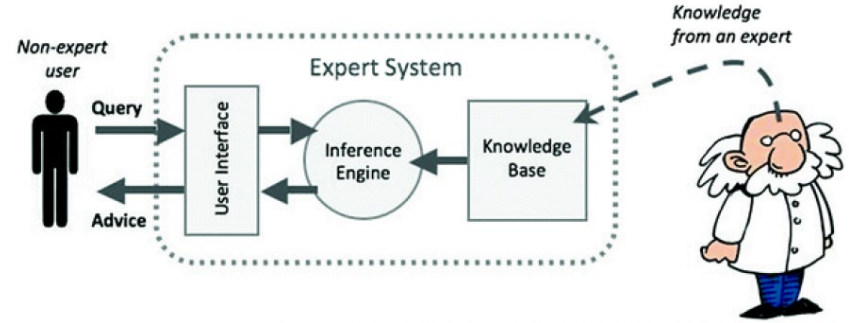


Artificial Intelligence, Machine & Deep Learning



AI: Rule-based learning

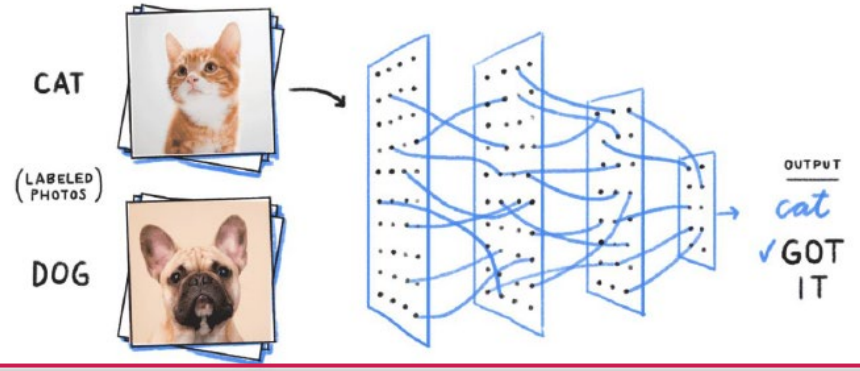
- Development of computer systems
- Performing tasks requiring human intelligence



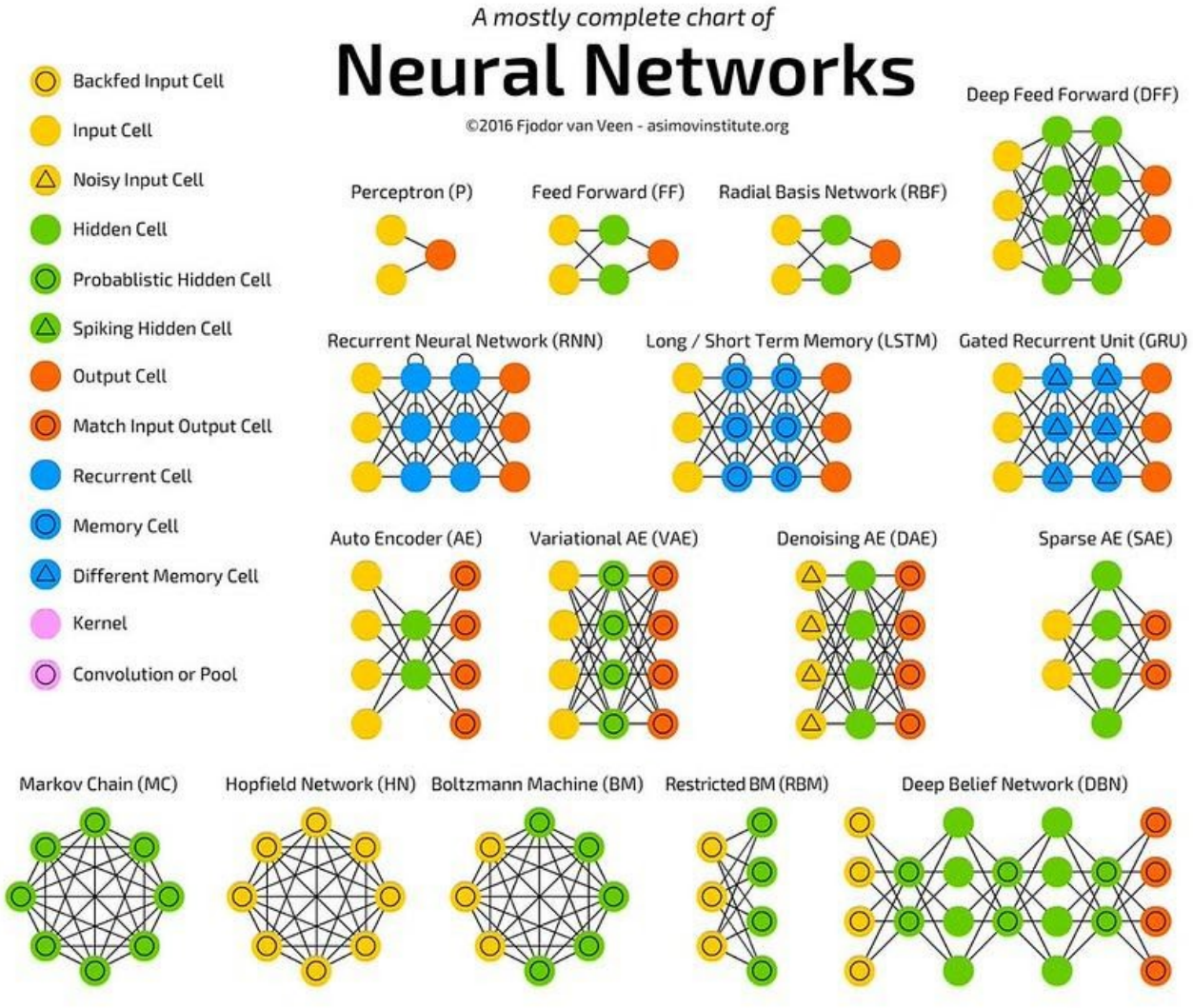
https://link.springer.com/chapter/10.1007/978-981-10-9035-6_82

ML: Learning from examples, human-made descriptors

DL: Deep neural networks, learning representations



It’s all Neural Networks …



- Fully connected layers
- Forward pass
- Loss function
- Backpropagation
- Regularization

Deep Learning / Neural Networks (NN)

- Extremely successful in large-data settings
- Can learn complex patterns
- Architecture depends on input (features)
  - Molecular fingerprint -> Multi-Layer Perceptron (MLP)/NN
  - Molecular graph -> Graph neural network (GNN)
  - SMILES -> Recurrent NN (RNN)/transformer

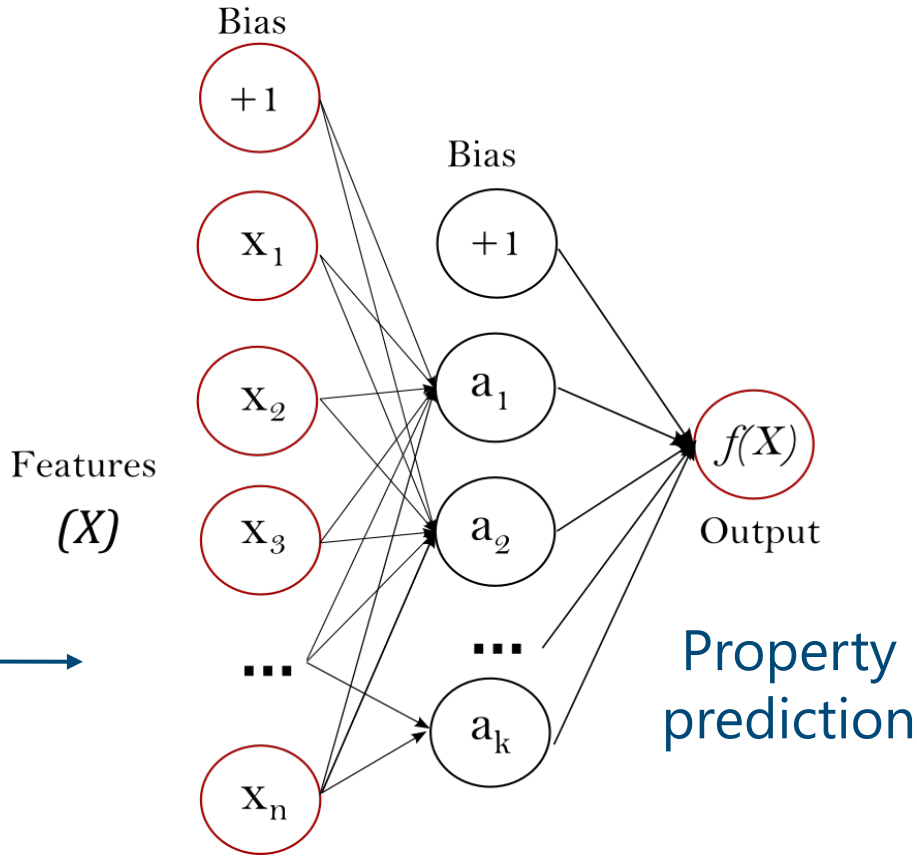
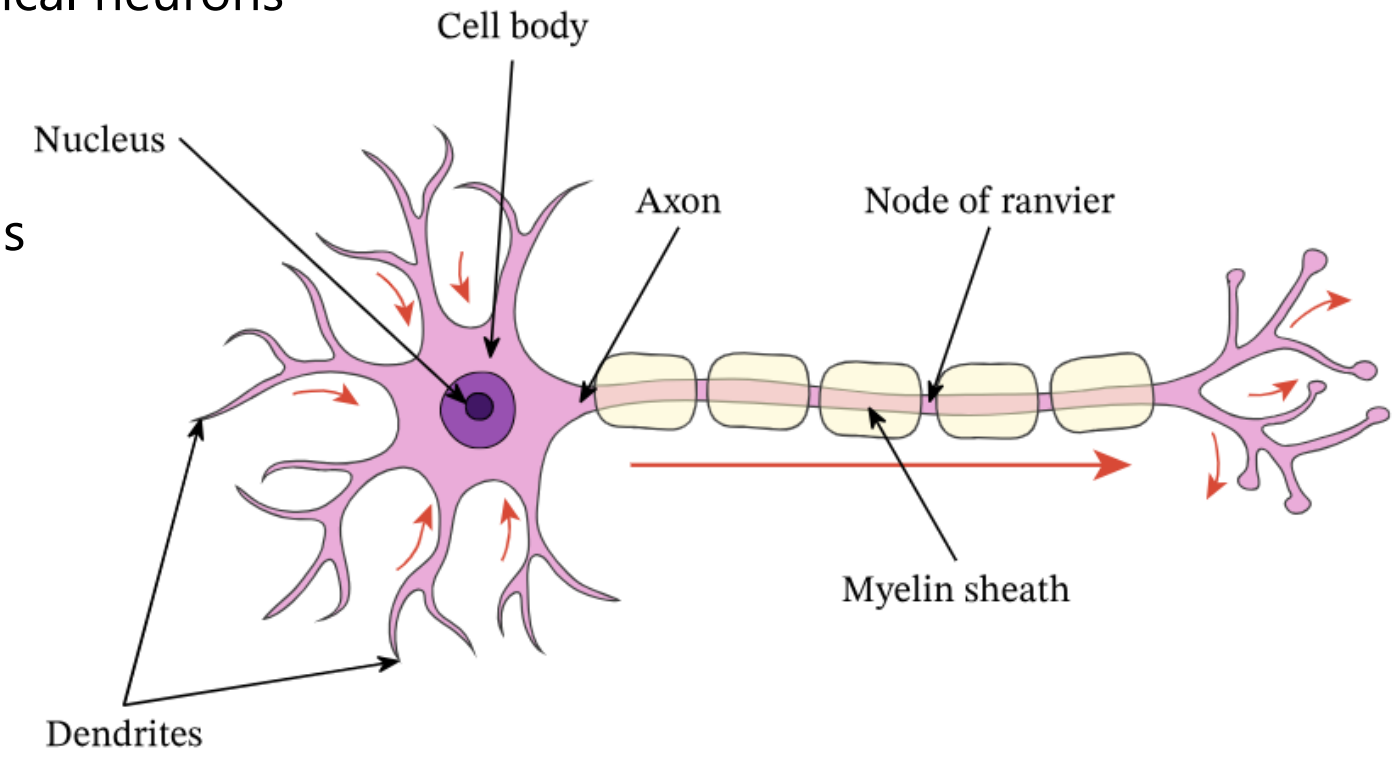


Figure from https://scikit-learn.org/stable/modules/neural_networks_supervised.html

Artificial Neural Networks

- Inspired by the human brain
- Mimick the signal transfer between biological neurons
  - Receives signals via dendrites
  - Processes signal in the cell body
  - Sends output via axon to other neurons



Structure of Neural Networks

Input layer: One neuron per feature in the input data

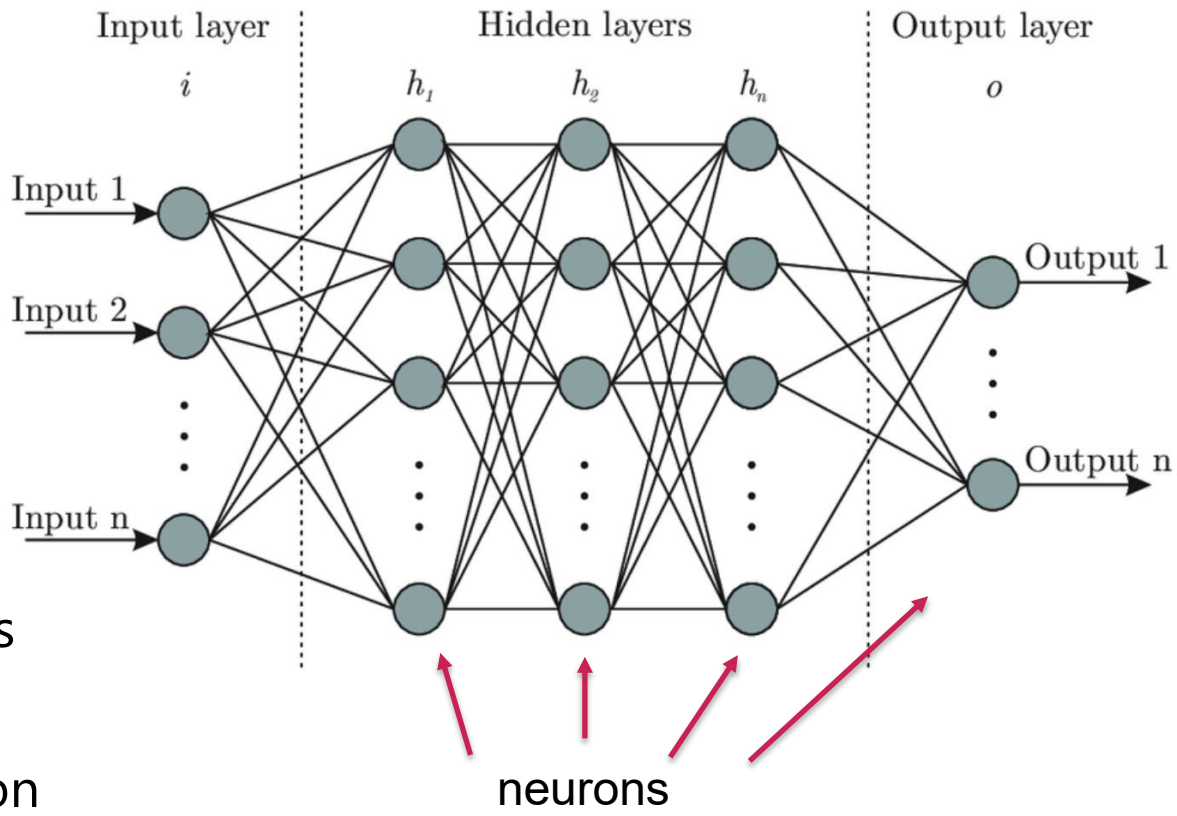
Hidden layers

- Depth of network
- (Often) same number of neurons over layers
- Learn intermediate representations

Output layer/neuron

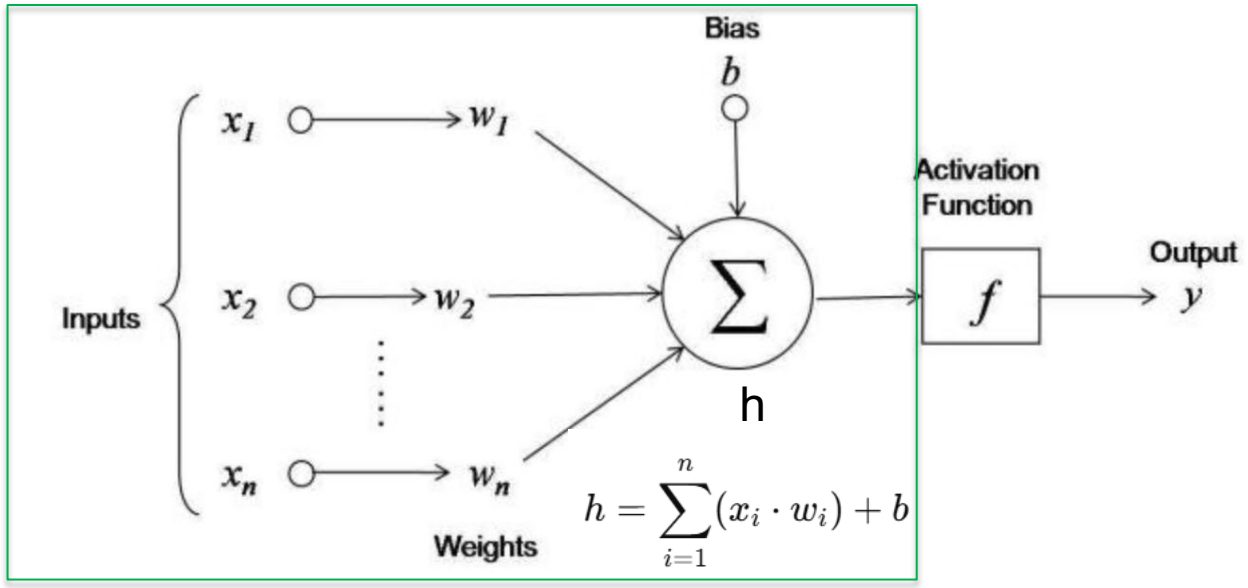
- Regression: Real number (e.g., pIC50 value)
- Binary classification: Probability of positive class

➢ Each layer transforms data to improve prediction



Neurons - Core Units of Neural Networks

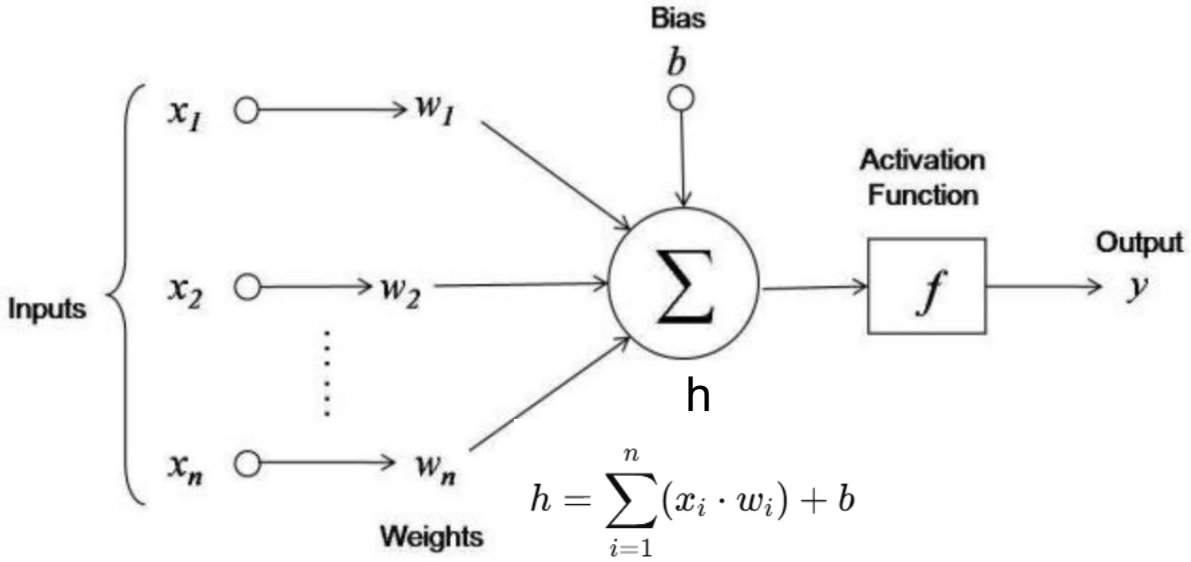
- Each input neuron (x_i) multiplied by a **weight** (w_i)
  - Weight determines the influence
- Multiplied values are summed
- **Bias** is added -> allows to shift the activation function
  - ➢ New value of the hidden neuron (h)



- Weights and biases are the **learnable parameters**
  - Randomly initialized
  - Tuned when training the model

Neurons - Core Units of Neural Networks

- **Activation function** applied to hidden neuron
  - Neuron is activated or not
- Activated neuron transmits information to the next layer
  - **Forward propagation**

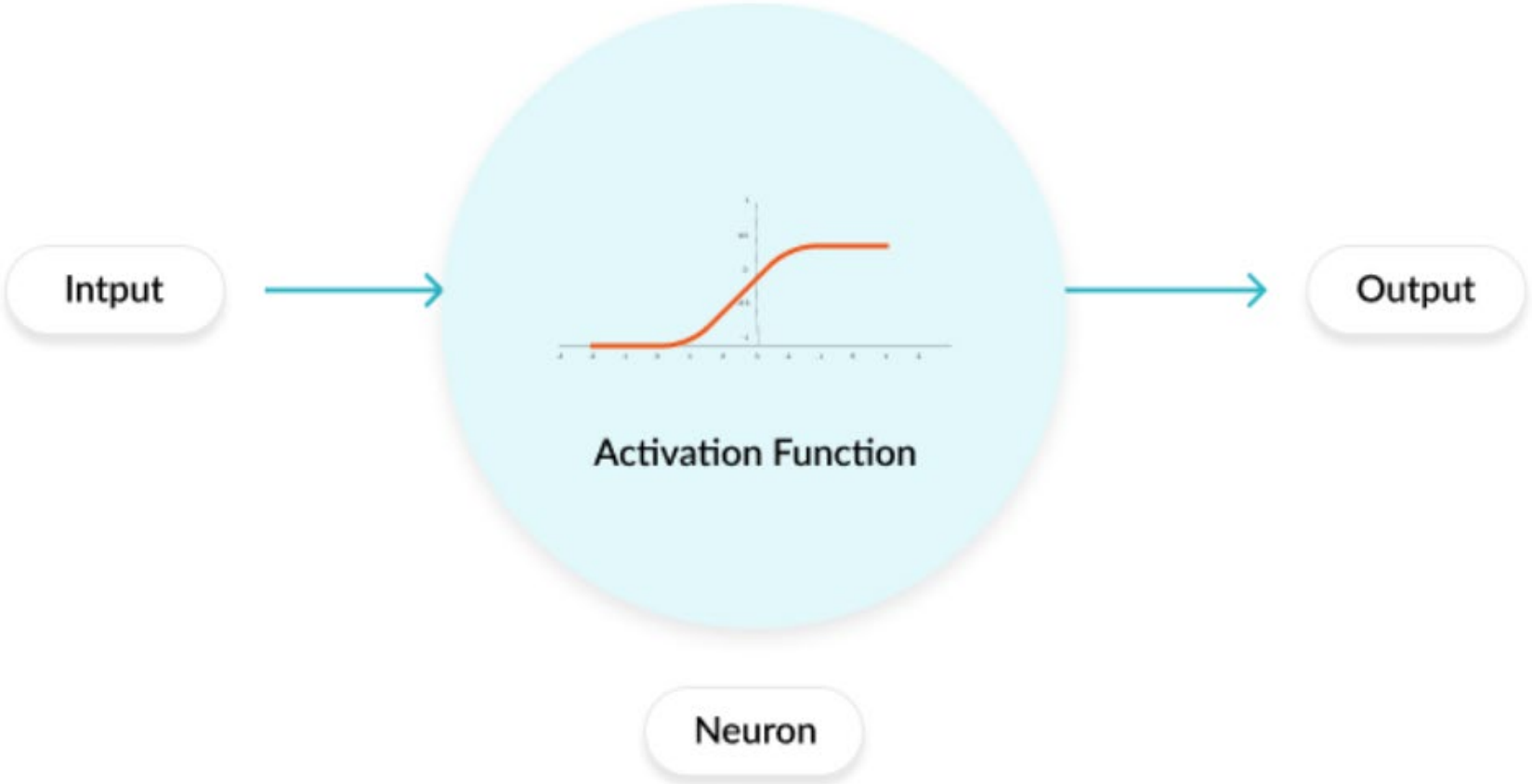


$$y = f\left(\sum_{i=1}^{n}(x_i \cdot w_i) + b\right)$$

- x_i = input values
- w_i = corresponding weights
- b = bias
- $f(\cdot)$ = activation function (e.g., ReLU, Sigmoid, Tanh)
- y = final output of the neuron

Activation Function

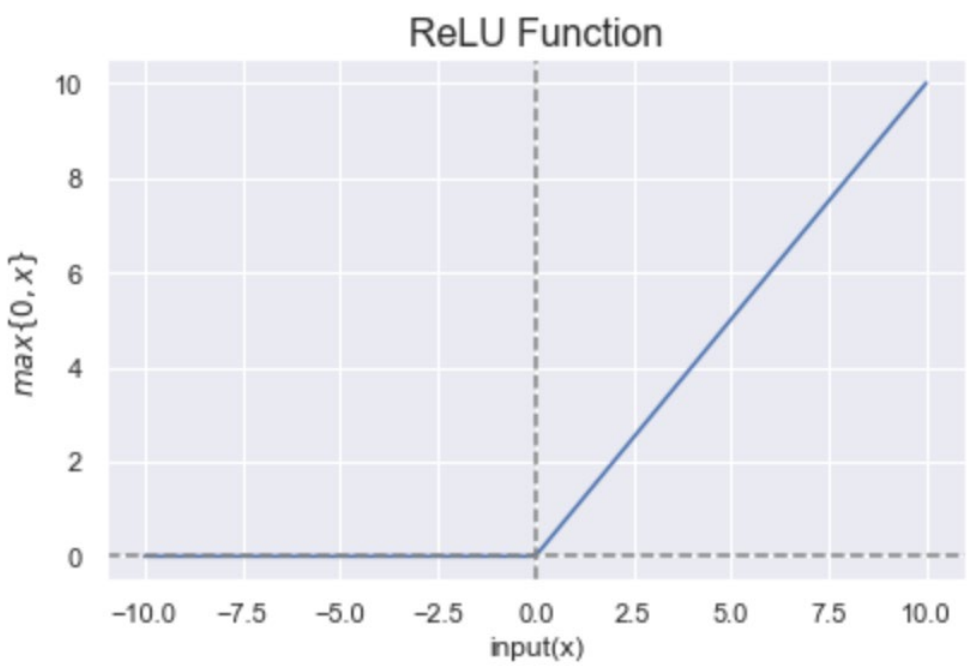
- Regulates the amount of information passed through the NN
- Applied to each neuron



Types of Activation Functions

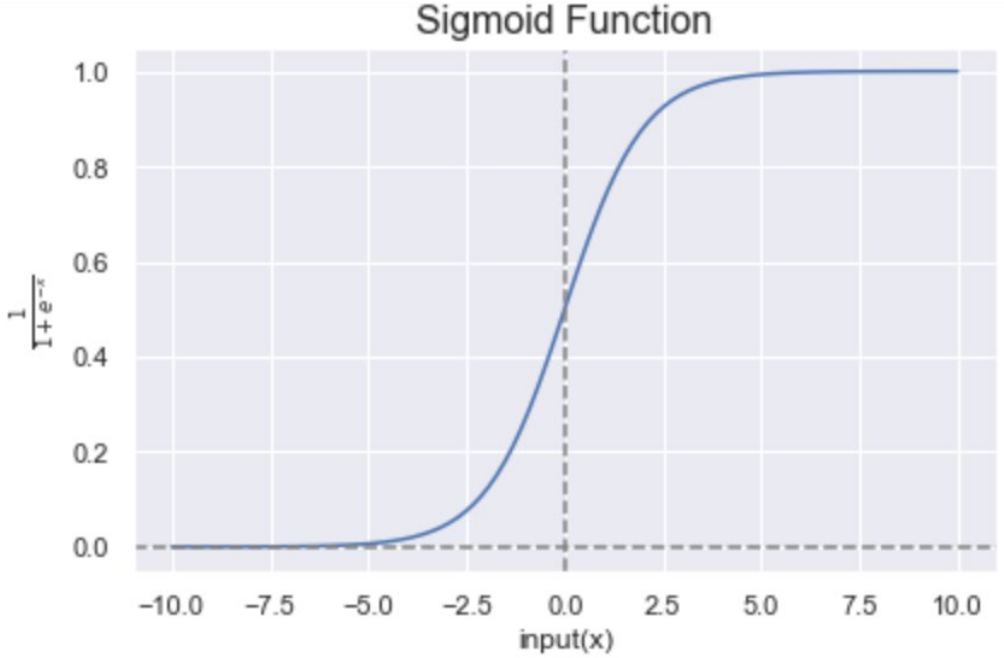
- Rectified Linear Unit (ReLU):
  - Commonly used because of its sparsity

$$f(x) = \max(0, x)$$



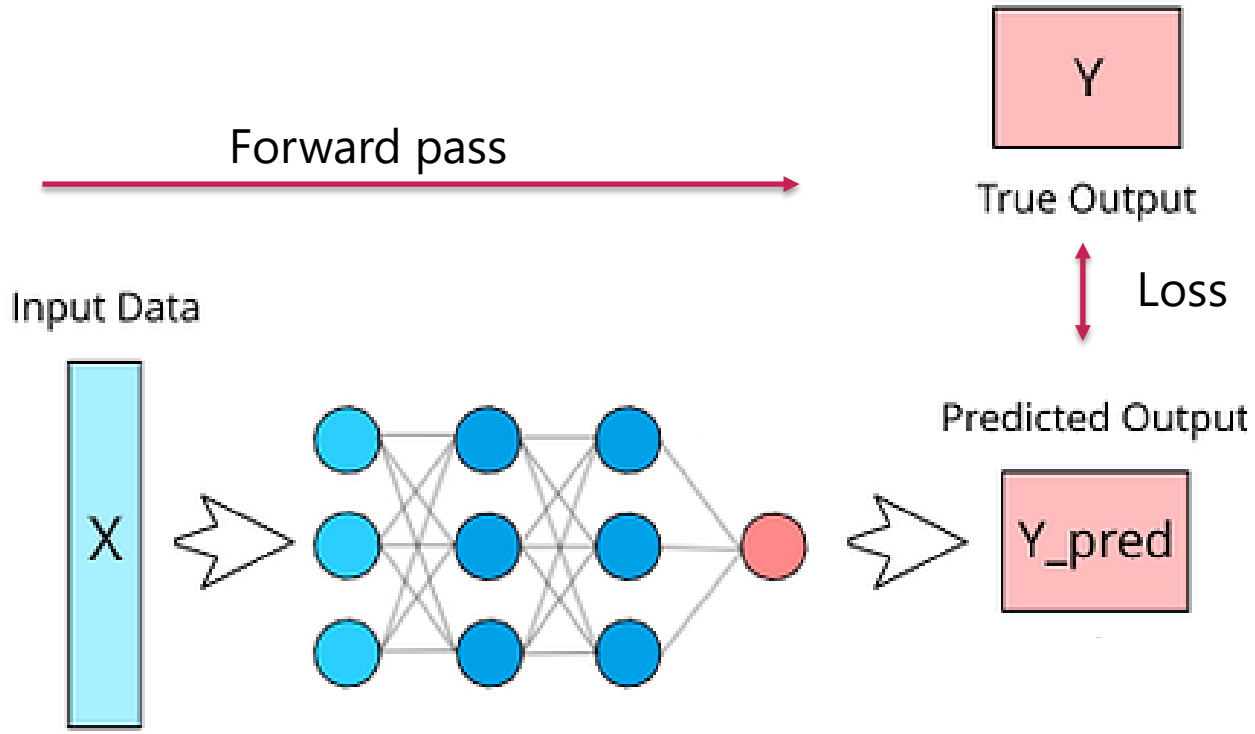
- Sigmoid function
  - Provides smooth gradient

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Training the Network Using a Loss Function

- Goal: Minimize the prediction error
  - **Loss function**: Difference between true value and predicted value
    - E.g. **MSE** (mean-squared-error)



- **Start**: Randomly initialized weights
  - ➢ High loss
  - ➢ How to find the optimal settings?

Training the Network Using a Loss Function

- Find the optimal set of parameters that minimize loss
  - Optimization algorithm: **Gradient descent** (GD)
    - Update the weights in the direction of the gradients to reduce the loss

  - Different types of gradient descent
    - **Batch** GD uses the entire dataset
      - Gathering all information before taking a single, well-calculated step
      - Accurate, but computationally very expensive
    - **Stochastic** GD uses one data point at a time
      - Fast, but more fluctuations in the path to the minimum
    - **Mini-batch** GD splits the dataset into small batches
      - Updates the parameters after each batch, balances accuracy and speed

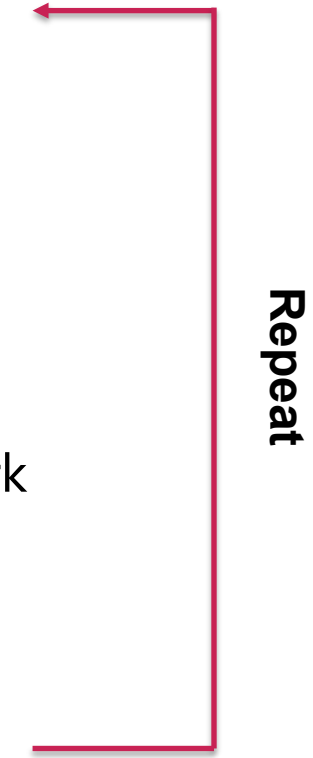
Training the Network Using a Loss Function

- **Start**: Randomly initialized weights → High loss
- Find the optimal set of parameters that minimize loss
  - Optimization algorithm: **Gradient descent** (GD)
    - Update the weights in the direction of the steepest decent (gradient of the loss)
    - **Learning rate**: Controls step size in weight updates
  - How much each weight contributed to the error: **Backpropagation**
    - Compute the gradients of the loss function w.r.t. each weight (using chain rule)

$$w_{\text{new}} = w_{\text{old}} - \alpha \cdot \frac{\partial L}{\partial w}$$

Training the Network – Step-by-Step Process

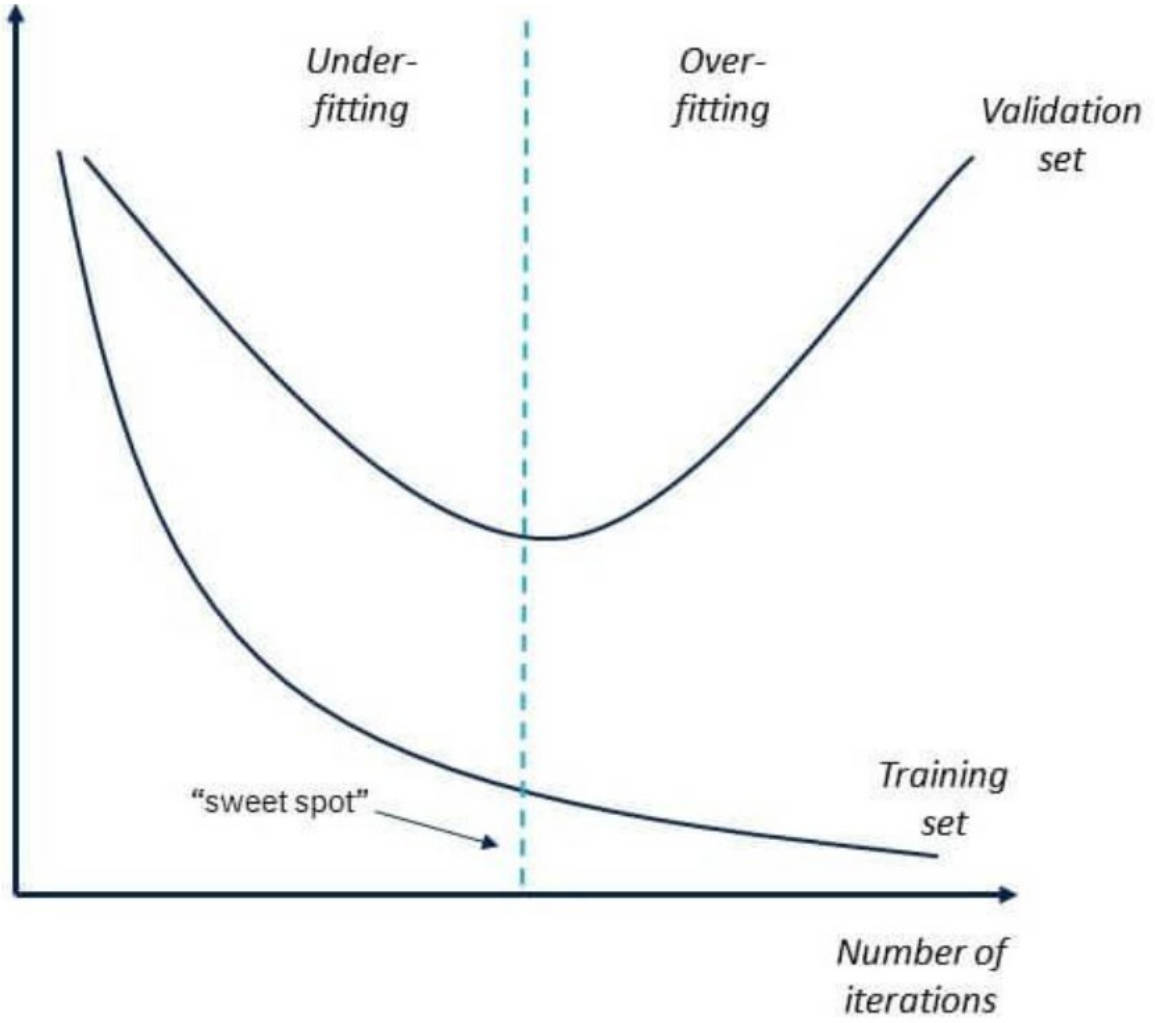
- **Forward pass**
  - Input data flows through the network
  - Network makes a prediction based on the current weights
- **Loss function**
  - Calculates the difference between the prediction and actual target value
- **Backward pass:** Backpropagation to compute gradients
  - Starts at the output layer and works its way backward through the network
  - Calculating the gradient of the loss with respect to each weight
  - Assigns *blame* for the error to each weight
- **Optimization:** Gradient descent to update weights
  - Adjust the weights in the direction that reduces the error



When to Stop Training?

Underfitting

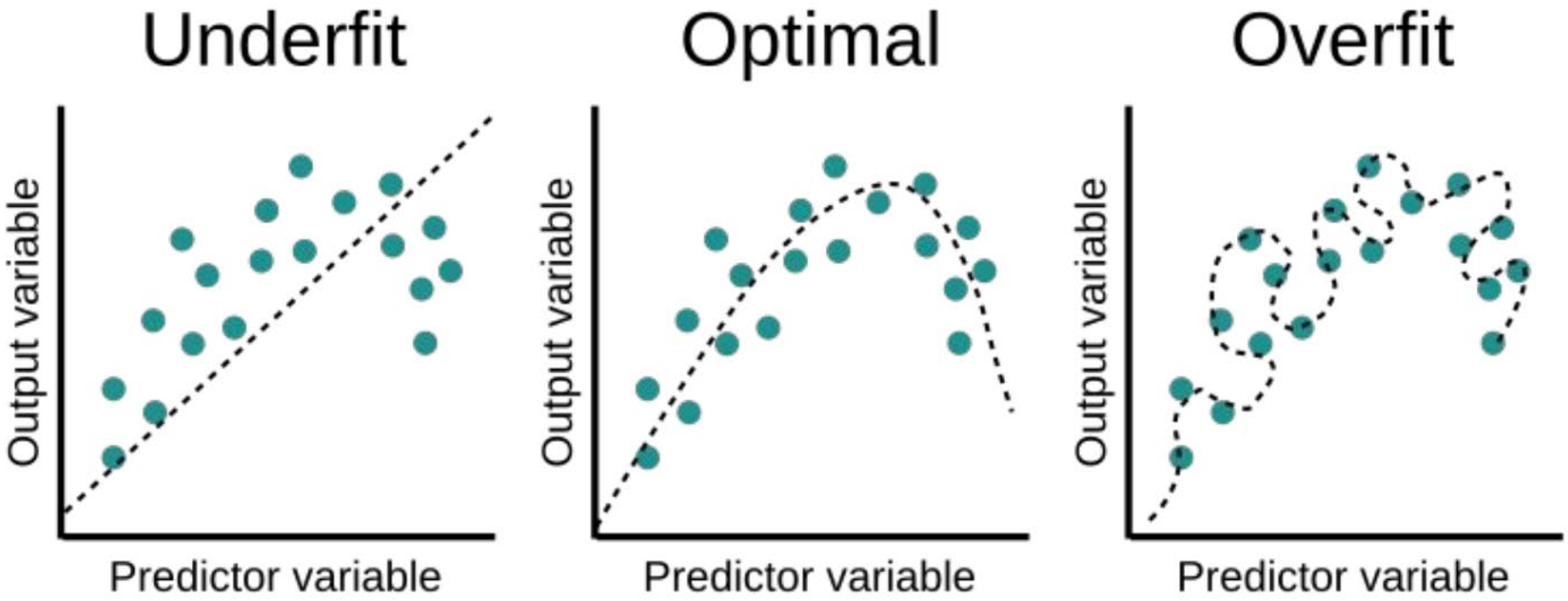
- Large training and validation errors
- Network did not learn
  - Too simple
  - Noisy data



Overfitting

- Small training, but large validation errors
- Network memorized the data
  - Too complex

When to Stop Training?



Challenges in Neural Network Training: Hyperparameters

- Number of hidden layers: Determines the depth of the network
  - Deeper networks can learn more complex patterns, but are harder to train
- Number of neurons per hidden layer: Affects the model's learning capacity
  - Too few may underfit, too many may overfit
- Activation functions: Different choices (ReLU, Sigmoid, Tanh, …) impact learning and convergence
- Optimization algorithm
  - Controls how weights are updated during training: Gradient descent, Adam, …
- Early stopping
  - Stops training when validation performance stops improving to prevent overfitting
- Dropout rate
  - Randomly disables neurons during training to improve generalization and reduce overfitting

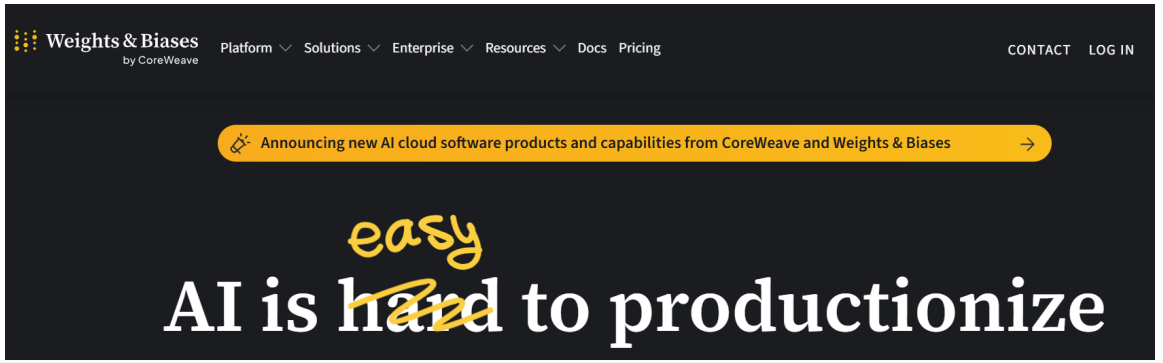
Validation Sets - Optimize Hyperparameters **without Cheating**

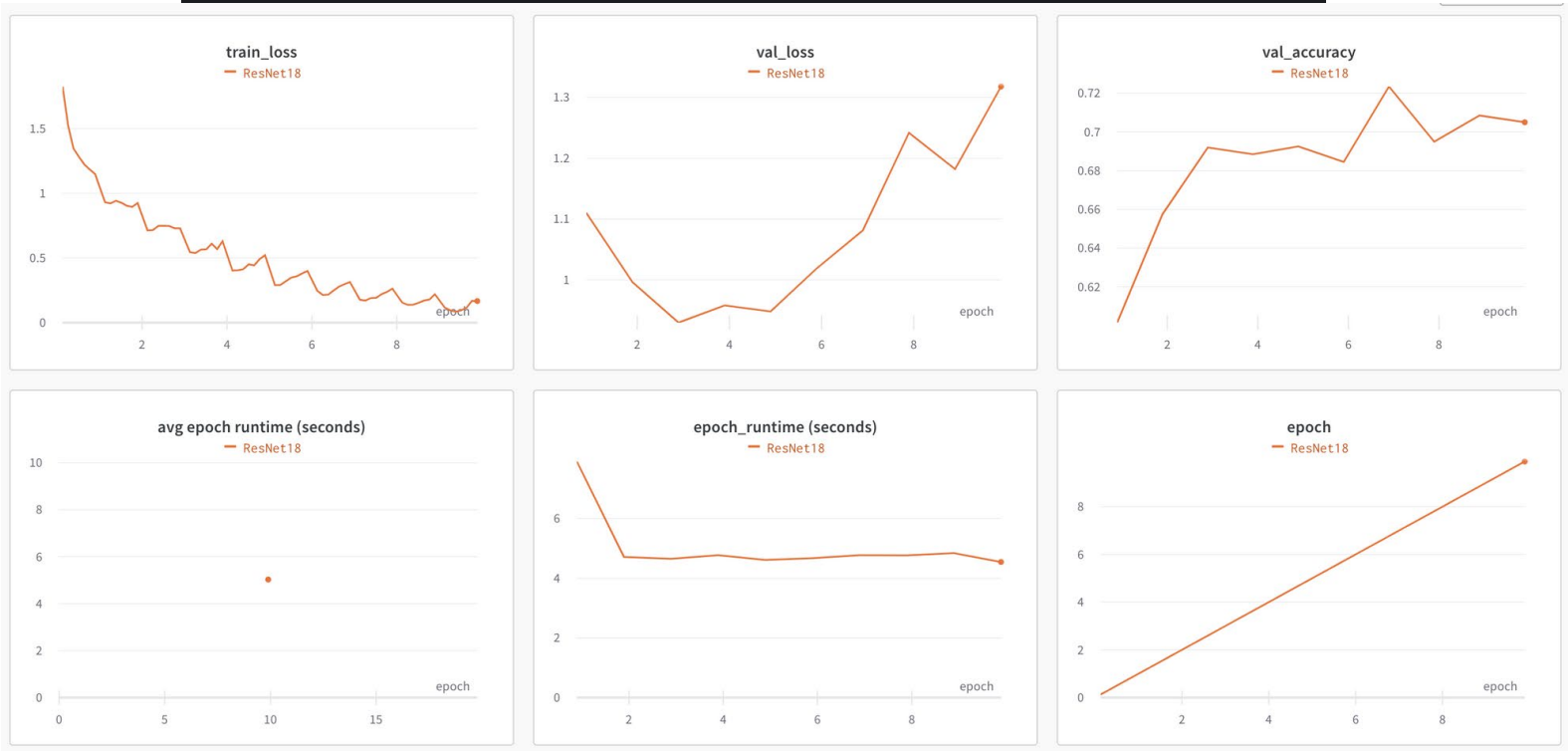
- Use a validation set alongside training and test set

- **Training**: Fitting the model
- **Validation**: Finding the best model*
- **Test**: Assess generalizability

*When to stop optimizing, which architecture to use, …

Nice Platform to Monitor DL Progress: https://wandb.ai/site





What we will do in detail …

Session 1

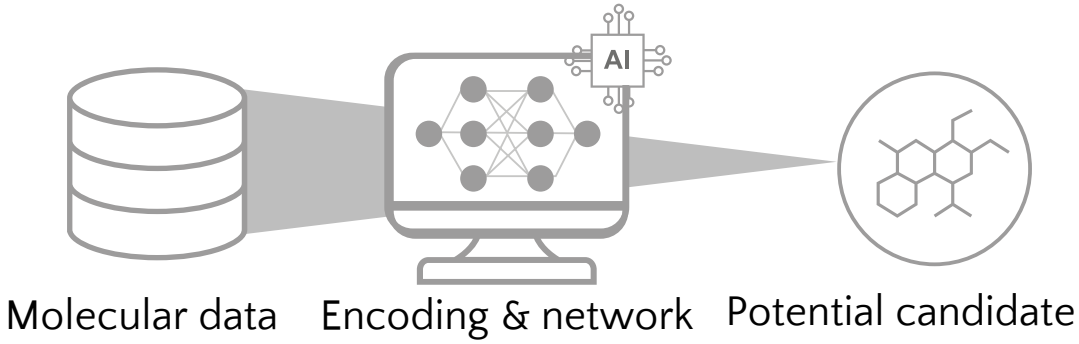
- CADD, Artificial Intelligence (AI) and Machine Learning (ML)
- The importance of data quality for AI/ML: Data preprocessing and analysis
- Molecular representations: Fingerprints and similarities

Session 2

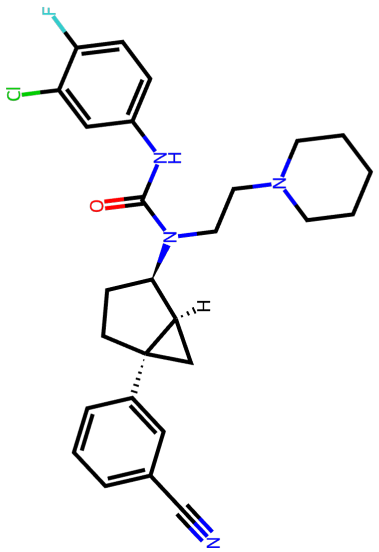
- Introduction machine learning (ML)
- (Un)supervised learning: RF, SVM, …
- Performance evaluation

Session 3

- Deep learning
- Neuronal networks
- **Molecular encodings and architectures**



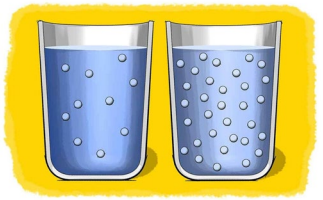
The Path From Molecules to Property Predictions

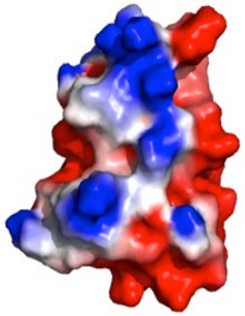


Supervised Learning (SL)

Representation
- Physchem properties
- Morgan FPs
- Adjacency matrix

Model
- RF
- SVM
- MLP/NN
- GNN





Self-Supervised Learning (SSL)

Tokenization
- Sequences
- Graphs

Foundational Models
- Transformers
- Autoencoders

Fine-tuning

Representation and Encoding of Molecules

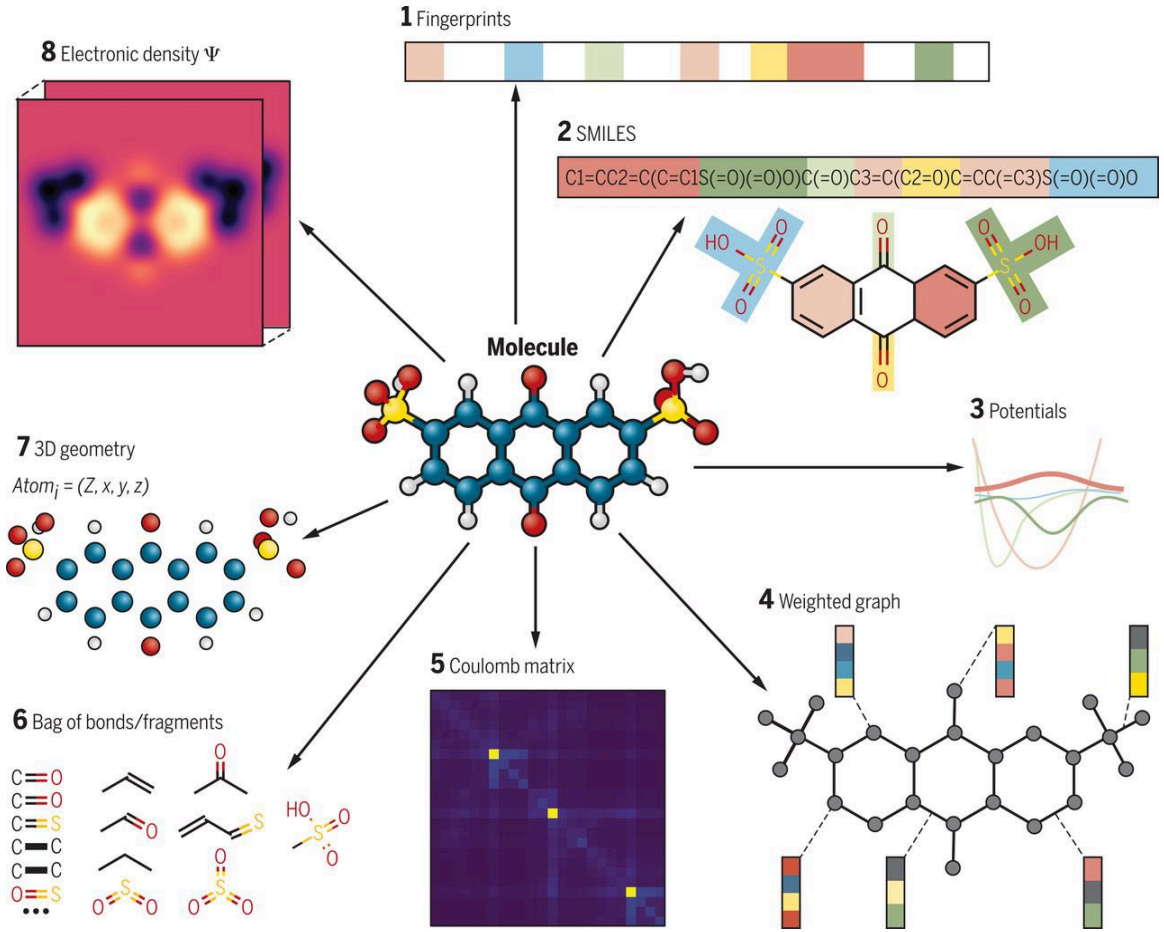
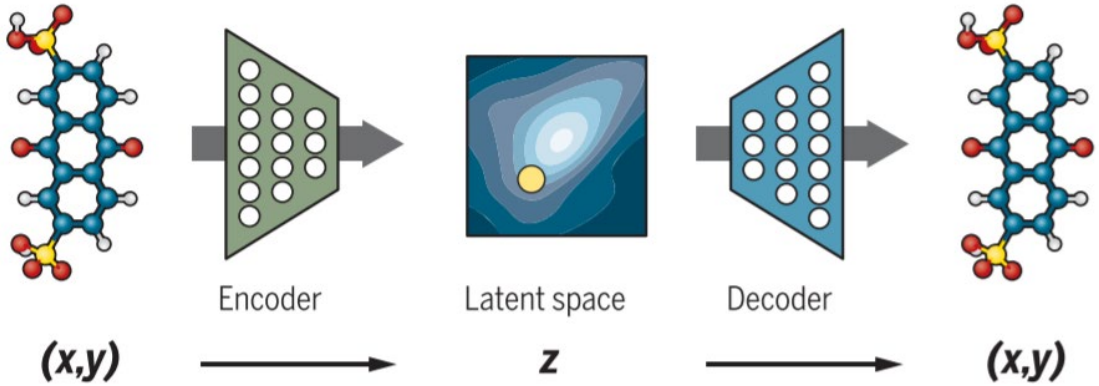


Figure taken from: Sanchez-Lengeling et al., Science 361, 360–365 (2018)

Learned representations: Embedding



SMILES strings paved the way for

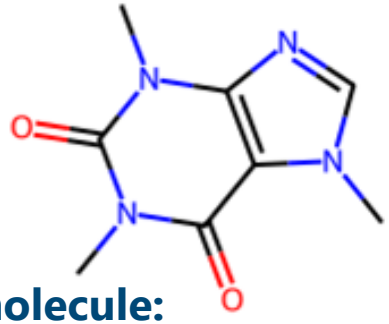
applying natural language processing techniques

to a broad range of molecule-related tasks

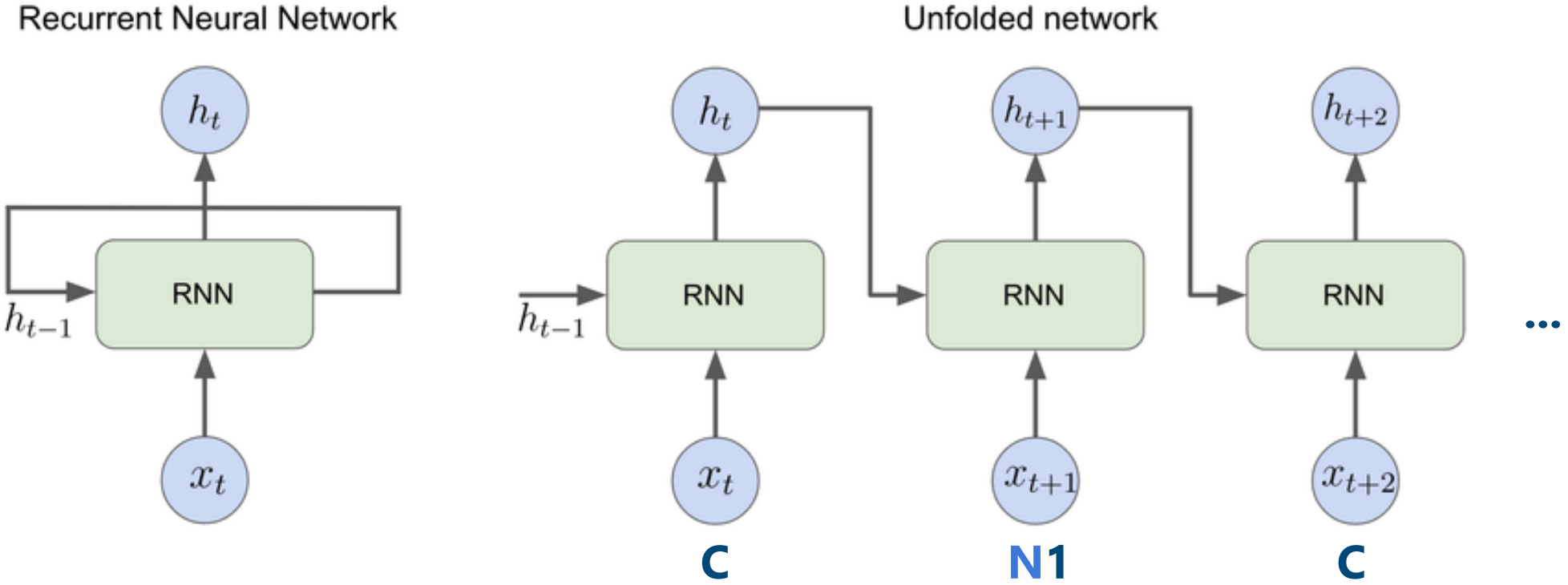
Sequence-Based Models on SMILES - RNNs

- Processes input sequentially (one time step after another).
- Maintains a hidden state that carries information from previous steps

→ Struggle to capture long-range interactions



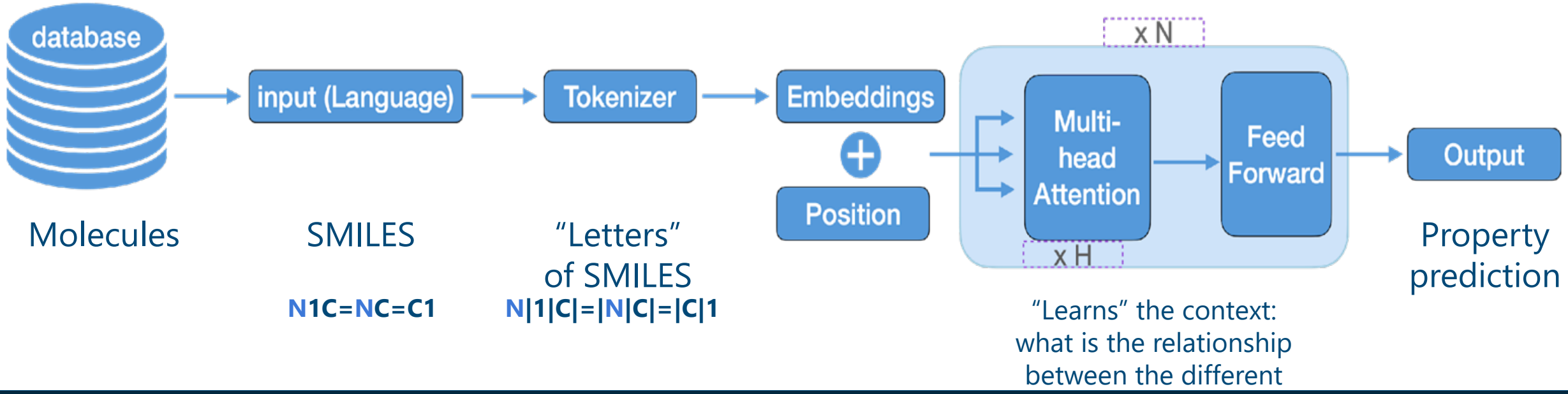
Input molecule:
CN1C=NC2=C1C(=O)N(C(=O)N2C)C



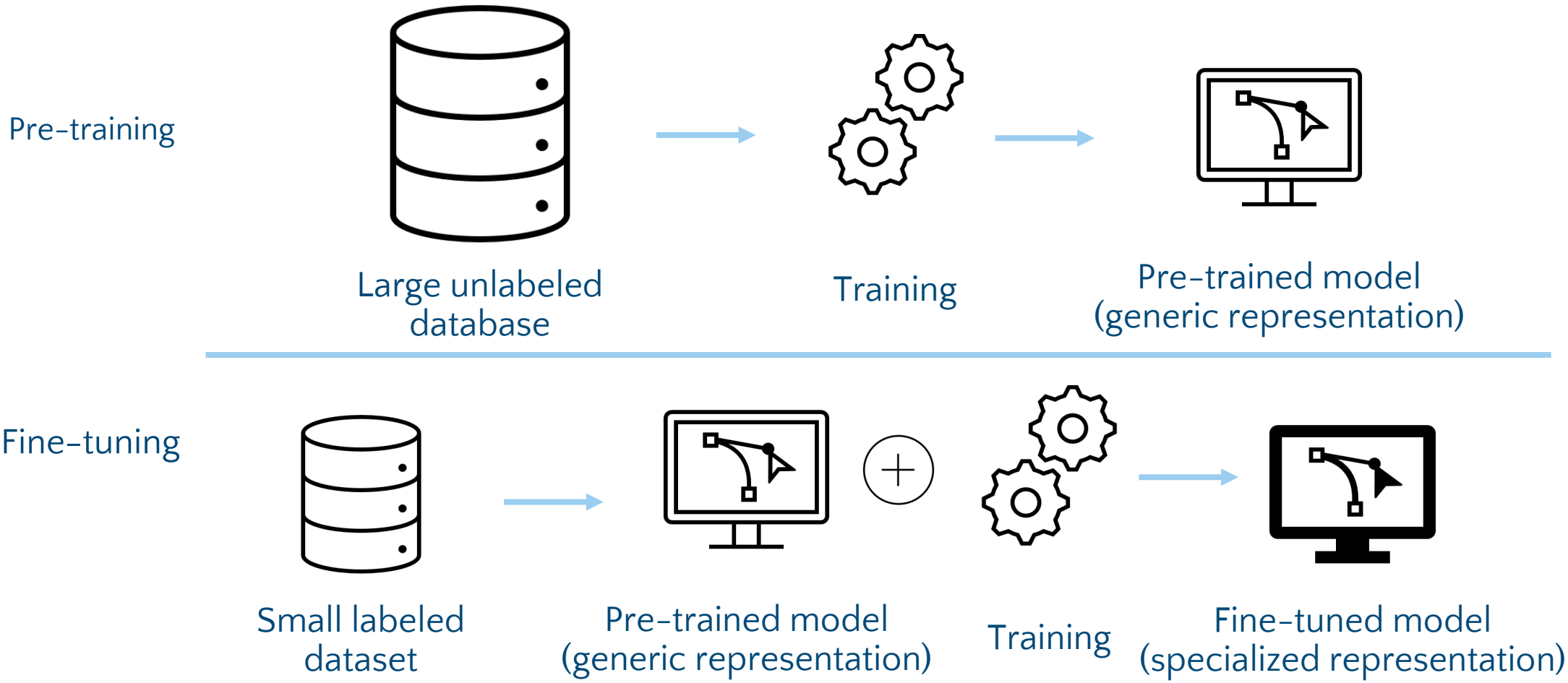
Sequence-Based Models on SMILES - Transformers

- Processes all tokens in parallel using self-attention
- No recurrence; uses positional encodings to maintain order

→ Transformers can be very powerful with enough data → pre-training



Sequence-Based Models on SMILES – Transformers



Feedback Very Welcome

https://forms.gle/KFya2kwpbTa9WTu47

Hands on ...

Time	Topic	Presenter
08:30-09:10	Lecture: From molecular data to models	Andrea Volkamer
09:10-10:00	Hands-on: Data	Katharina Buchthal
10:00-10:40	Lecture: Machine learning (ML) essentials	Andrea Volkamer
10:40-11:30	Hands-on: ML	Katharina Buchthal
11:30-12:30	Break	
12:30-13:00	Hands-on: continued	Katharina Buhcthal
13:00-13:40	Lecture: Deep learning (DL) applications	Andrea Volkamer
13:40-14:30	Hands-on: DL	Katharina Buchthal

https://github.com/volkamerlab/cic_summerschool_2025